

React Tic-Tac-Toe Tutorial: Hinglish Explanation

Ye tutorial Tic-Tac-Toe game banate hue React ke fundamental concepts sikhata hai. Iska main purpose sirf game banana nahi, balki ye samajhna hai ki **components, props, state, events, immutability, lifting state up, lists, keys aur derived state** actual app mein milkar kaise kaam karte hain.

1. Final application kya karti hai?

Tutorial complete karne ke baad game:

- 3×3 Tic-Tac-Toe board dikhata hai
- Player ko X aur O turns alternate karke deta hai
- Filled square ko overwrite hone se rokta hai
- Winner identify karta hai
- Har move ki history save karta hai
- User ko kisi bhi previous move par jump karne deta hai
- Past move par jaakar nayi game branch start karne deta hai

Final app mein teen main functions/components hain:

```
1 Square
2 Board
3 Game
```

Aur winner check karne ke liye:

```
1 calculateWinner()
```

Part 1: Initial setup

Tutorial browser mein CodeSandbox use karne ka option deta hai. Starter code mein mainly ye files hoti hain:

- App.js
- styles.css
- index.js
- public folder

Local environment mein project chalane ke liye page par diye gaye steps hain:

1. Node.js install karo.

2. CodeSandbox project download karo.
3. Archive unzip karo.
4. Terminal se project folder mein jao.
5. Dependencies install karo:

```
1 npm install
```

1. Development server run karo:

```
1 npm start
```

Part 2: React component kya hota hai?

Starting component:

```
1 export default function Square() {  
2   return <button className="square">X</button>;  
3 }
```

Is code ka meaning

function Square()

Ye Square naam ka React component hai. Component basically ek JavaScript function hai jo UI describe karta hai.

export default

Isse doosri file is component ko import kar sakti hai. default batata hai ki ye file ka primary export hai.

JSX

```
1 <button className="square">X</button>
```

Ye JSX hai. JSX HTML jaisa dikhta hai, lekin JavaScript ke andar likha jata hai.

React mein CSS class dene ke liye:

```
1 className="square"
```

likhte hain, normal HTML ka class nahi.

Part 3: Multiple JSX elements aur Fragment

React component directly multiple adjacent elements return nahi kar sakta:

```
1 return <button>X</button><button>X</button>;
```

Isliye unhe ek parent element mein wrap karna padta hai. Extra DOM element nahi chahiye ho to React Fragment use kar sakte ho:

```
1 return (  
2   <>  
3     <button>X</button>  
4     <button>X</button>  
5   </>  
6 );
```

<>...</> ko **Fragment** kehte hain.

Iska kaam multiple JSX elements ko group karna hai.

Part 4: Board banana

Tic-Tac-Toe board ke liye 9 buttons ko 3 rows mein divide kiya gaya:

```
1 <div className="board-row">  
2   <button className="square">1</button>  
3   <button className="square">2</button>  
4   <button className="square">3</button>  
5 </div>
```

Aisi total 3 rows banayi gayi hain.

Initially component ka naam Square tha, lekin jab uske andar poora board aa gaya to naam Board kar diya:

```
1 export default function Board() {  
2   // Board JSX  
3 }
```

Ye naming ka important lesson hai: **component ka naam uske actual responsibility ko represent karna chahiye.**

Part 5: Reusable Square component

Har button manually repeat karne se duplicated code banta. Isliye reusable Square component create kiya:

```
1 function Square() {
```

```
2   return <button className="square">1</button>;
3 }
```

Phir Board ke andar:

```
1 <Square />
2 <Square />
3 <Square />
```

Custom React components ka naam capital letter se start hona chahiye:

```
1 <Square />
```

Lowercase names jaise <button> aur <div> built-in browser elements ko represent karte hain.

Part 6: Props se data pass karna

Problem ye thi ki sabhi squares 1 show kar rahe the. Har Square ko alag value dene ke liye props use hue.

Parent component:

```
1 <Square value="1" />
2 <Square value="2" />
3 <Square value="3" />
```

Child component:

```
1 function Square({ value }) {
2   return <button className="square">{value}</button>;
3 }
```

Props ka flow

```
1 Board → value prop → Square
```

Board parent hai aur Square child hai.

Curly braces kyun?

Agar likhoge:

```
1 <button>value</button>
```

to screen par literal value text dikhega.

JavaScript variable ki actual value show karne ke liye:

```
1 <button>{value}</button>
```

JSX mein {} ka meaning hai: **ab iske andar JavaScript expression evaluate karo.**

Part 7: Click event handle karna

Square click detect karne ke liye event handler banaya:

```
1 function handleClick() {  
2   console.log('clicked!');  
3 }
```

Button ke saath attach kiya:

```
1 <button onClick={handleClick}>  
2   {value}  
3 </button>
```

Dhyan do:

```
1 onClick={handleClick}
```

mein function pass ho raha hai, immediately call nahi ho raha.

Galat pattern:

```
1 onClick={handleClick()}
```

Ismein function rendering ke samay hi run ho jayega.

Part 8: useState se component ko memory dena

Component ko yaad rakhna tha ki Square click hua hai. React mein component ko data remember karane ke liye state use hoti hai.

```
1 import { useState } from 'react';
```

Square ke andar:

```
1 const [value, setValue] = useState(null);
```

Is line mein:

- value current state hai
- setValue state update karne wala function hai
- null initial value hai

Click hone par:

```
1 function handleClick() {  
2   setValue('X');  
3 }
```

Complete component:

```
1  function Square() {
2    const [value, setValue] = useState(null);
3
4    function handleClick() {
5      setValue('X');
6    }
7
8    return (
9      <button className="square" onClick={handleClick}>
10        {value}
11      </button>
12    );
13  }
```

Jab setValue('X') call hota hai:

1. State update hoti hai.
2. React component ko re-render karta hai.
3. Updated value screen par दिखाई देता है.

Har Square instance ka apna independent state tha. Isliye kisi ek Square ko click karne se doosre Square ki value change nahi hoti.

Part 9: Lifting state up

Individual Square state basic example ke liye sahi thi, lekin winner calculate karne ke liye Board ko sabhi 9 squares ki values maloom honi chahiye.

Isliye state ko individual Square se hata kar common parent Board mein move kiya gaya. Is technique ko **lifting state up** kehte hain.

```
1  const [squares, setSquares] = useState(Array(9).fill(null));
```

Ye initially aisa array banata hai:

```
1  [
2    null, null, null,
3    null, null, null,
4    null, null, null
5  ]
```

Har index board ke ek Square ko represent karta hai:

```
1  0 1 2
2  3 4 5
3  6 7 8
```

Board values child components ko pass karta hai:

```
1  <Square value={squares[0]} />
2  <Square value={squares[1]} />
3  <Square value={squares[2]} />
```

Ab Square ko apna state maintain karne ki zarurat nahi:

```
1  function Square({ value }) {
2    return <button className="square">{value}</button>;
3  }
```

Lifting state up ka main rule

Agar multiple child components ko shared data chahiye ya unke beech coordination chahiye, to state unke nearest common parent mein rakho.

Part 10: Child parent ki state kaise update karega?

State Board ke paas hai, lekin button click Square ke andar hota hai.

Child directly parent ki private state update nahi karta. Parent ek function prop ke through child ko deta hai.

Square:

```
1  function Square({ value, onSquareClick }) {
2    return (
3      <button className="square" onClick={onSquareClick}>
4        {value}
5      </button>
6    );
7  }
```

Board:

```
1  <Square
2    value={squares[0]}
3    onSquareClick={() => handleClick(0)}
4  />
```

Iska data flow:

```
1 Board state
2   ↓ value prop
3 Square
4   ↓ user click
5 onSquareClick
6   ↓
7 Board ka handleClick()
8   ↓
9 Board state update
```

Isse React ka **one-way data flow** samajh aata hai:

- Data parent se child ki taraf props ke through jata hai.
 - Child events ke through parent ko update request deta hai.
-

Part 11: Arrow function kyun use hui?

Ye code problematic hai:

```
1 onSquareClick={handleClick(0)}
```

Ismein handleClick(0) render ke samay immediately call hoga. Agar uske andar state update hui to component re-render hoga, function phir chalega aur infinite loop ho sakti hai.

Isliye function ko delayed manner mein pass kiya:

```
1 onSquareClick={() => handleClick(0)}
```

handleClick(0) ab tabhi chalega jab click event hoga.

Difference:

```
1 handleClick          // function pass karna
2 handleClick(0)       // function immediately call karna
3 () => handleClick(0) // wrapper function pass karna
```

Part 12: State ko immutable rakhna

Square update karte waqt original array ko directly modify nahi kiya:

```
1 const nextSquares = squares.slice();
2 nextSquares[i] = 'X';
3 setSquares(nextSquares);
```

squares.slice() ek shallow copy banata hai.

Direct mutation hoti:

```
1 squares[i] = 'X';
```

Tutorial is approach ko avoid karta hai.

Immutability ke benefits

1. Previous data safe rehta hai

Original board state overwrite nahi hota. Isi wajah se later game history aur time travel possible hota hai.

2. Changes compare karna easy hota hai

React ke liye old aur new references compare karna easier hota hai.

3. Undo/redo jaisi features easy hoti hain

Har state ka separate snapshot maintain kiya ja sakta hai.

Part 13: X aur O ke turns

Ye track karne ke liye initially state add ki gayi:

```
1 const [xIsNext, setXIsNext] = useState(true);
```

Move ke samay:

```
1 if (xIsNext) {  
2   nextSquares[i] = 'X';  
3 } else {  
4   nextSquares[i] = 'O';  
5 }
```

Phir turn reverse:

```
1 setXIsNext(!xIsNext);
```

Isse moves alternate hote hain:

```
1 X → O → X → O
```

Part 14: Filled square overwrite hone se rokna

Agar user same Square ko dobara click kare to existing value overwrite nahi honi chahiye.

```
1 if (squares[i]) {
```

```
2   return;  
3 }
```

Agar squares[i] mein already X ya O hai, function early return kar deta hai.

Ye **guard clause** ya **early return** ka example hai.

Part 15: Winner calculate karna

Possible winning combinations:

```
1  const lines = [  
2    [0, 1, 2],  
3    [3, 4, 5],  
4    [6, 7, 8],  
5    [0, 3, 6],  
6    [1, 4, 7],  
7    [2, 5, 8],  
8    [0, 4, 8],  
9    [2, 4, 6],  
10   ];
```

Har combination ke indexes destructure kiye jaate hain:

```
1  const [a, b, c] = lines[i];
```

Winner condition:

```
1  if (  
2    squares[a] &&  
3    squares[a] === squares[b] &&  
4    squares[a] === squares[c]  
5  ) {  
6    return squares[a];  
7  }
```

Meaning:

1. First position empty na ho.
2. First aur second value same hon.
3. First aur third value same hon.

Agar match mila to 'X' ya 'O' return hota hai. Warna:

```
1  return null;
```

Game jeetne ke baad new move ko stop kiya:

```
1  if (calculateWinner(squares) || squares[i]) {  
2    return;  
3  }
```

Part 16: Game status show karna

```
1  const winner = calculateWinner(squares);  
2  
3  let status;  
4  
5  if (winner) {  
6    status = 'Winner: ' + winner;  
7  } else {  
8    status = 'Next player: ' + (xIsNext ? 'X' : 'O');  
9  }
```

JSX:

```
1  <div className="status">{status}</div>
```

Ternary expression:

```
1  xIsNext ? 'X' : 'O'
```

Meaning:

- xIsNext true hai to 'X'
- warna 'O'

Part 17: Time travel ke liye history

Har move ke baad board array ki nayi copy banti hai. Isliye har board state ko history array mein store kiya ja sakta hai.

History roughly aisi hoti hai:

```
1  [  
2    [null, null, null, null, null, null, null, null, null],  
3    [null, null, null, null, 'X', null, null, null, null],  
4    [null, null, null, null, 'X', null, null, null, 'O']  
5  ]
```

Har inner array kisi particular move ke baad poore board ka snapshot hai.

Initial history:

```
1  const [history, setHistory] = useState([
2    Array(9).fill(null)
3  ]);
```

Dhyan rahe:

```
1  Array(9).fill(null)
```

single board hai, lekin:

```
1  [Array(9).fill(null)]
```

ek history array hai jiske andar initial board hai.

Part 18: State ko dobara lift karna

Pehle state Square se Board mein lift hui thi. Ab history manage karne ke liye state Board se Game component mein lift hoti hai.

Final hierarchy:

```
1  Game
2    └─ Board
3        └─ Square × 9
```

Responsibilities:

Square

- Ek button render karta hai
- Value display karta hai
- Click callback run karta hai

Board

- 9 Squares render karta hai
- Click ke baad next board create karta hai
- Winner aur current status calculate karta hai

Game

- Complete game history maintain karta hai
 - Current move maintain karta hai
 - Time travel control karta hai
 - Board ko current snapshot deta hai
-

Part 19: Controlled Board

Final version mein Board apna state maintain nahi karta. Use required information props ke through milti hai:

```
1 function Board({ xIsNext, squares, onPlay }) {
```

Props:

- xIsNext: next player kaun hai
- squares: currently displayed board
- onPlay: new move parent ko bhejne ka function

Board new state directly save nahi karta:

```
1 onPlay(nextSquares);
```

Game mein:

```
1 <Board
2   xIsNext={xIsNext}
3   squares={currentSquares}
4   onPlay={handlePlay}
5 />
```

Is context mein Board ko controlled component samajh sakte ho, kyunki uska important data parent Game control kar raha hai.

Part 20: History update karna

New move add karne ke liye:

```
1 function handlePlay(nextSquares) {
2   const nextHistory = [
3     ...history.slice(0, currentMove + 1),
4     nextSquares
5   ];
6
7   setHistory(nextHistory);
8   setCurrentMove(nextHistory.length - 1);
9 }
```

`history.slice(0, currentMove + 1)`

Ye history ko current selected move tak rakhta hai.

Maan lo history mein 6 moves hain. User move 2 par wapas gaya aur wahan se naya move khela. Purane moves 3–6 ab remove ho jayenge aur move 2 se nayi timeline create hogi.

Spread syntax

```
1 [...oldHistory, nextSquares]
```

Iska meaning:

- Old history ke saare elements copy karo
 - End mein new board add karo
-

Part 21: Current move track karna

```
1 const [currentMove, setCurrentMove] = useState(0);
```

Current board:

```
1 const currentSquares = history[currentMove];
```

Agar currentMove:

- 0 hai, initial empty board dikhega
 - 1 hai, first move ke baad ka board dikhega
 - 2 hai, second move ke baad ka board dikhega
-

Part 22: History list ke liye map

History ke har item ko button mein convert kiya:

```
1 const moves = history.map((squares, move) => {
2   let description;
3
4   if (move > 0) {
5     description = 'Go to move #' + move;
6   } else {
7     description = 'Go to game start';
8   }
9
10  return (
11    <li key={move}>
12      <button onClick={() => jumpTo(move)}>
13        {description}
```

```
14     </button>
15   </li>
16 );
17 });
```

map() ek array ko doosre array mein transform karta hai.

Yahan:

```
1  history snapshots
2      ↓ map()
3  React list items aur buttons
```

Note: Callback ka squares parameter final code mein use nahi ho raha, lekin woh current history entry ko represent karta hai. move uska index hai.

Part 23: React list mein key

Dynamic list render karte waqt har sibling element ko unique key deni chahiye:

```
1 <li key={move}>
```

Key React ko batati hai ki kaunsa list item kaunsi identity represent karta hai.

Normally array index ko key banana risky ho sakta hai agar items:

- Reorder hon
- Insert hon
- Delete hon

Lekin is tutorial mein move number sequential aur stable identity hai. Moves reorder nahi hote, isliye:

```
1 key={move}
```

safe hai.

Important: key normal prop nahi hai. React ise internally use karta hai aur child component ise usual prop ki tarah access nahi kar sakta.

Part 24: Time travel implement karna

```
1 function jumpTo(nextMove) {
2   setCurrentMove(nextMove);
3 }
```

History button click:

```
1 <button onClick={() => jumpTo(move)}>
```

Button click hone par:

1. currentMove update hota hai.
2. currentSquares = history[currentMove] new snapshot select karta hai.
3. Component re-render hota hai.
4. Board selected past position display karta hai.

History khud delete nahi hoti, sirf displayed snapshot change hota hai.

Part 25: Redundant state remove karna

Initially xIsNext separate state thi:

```
1 const [xIsNext, setXIsNext] = useState(true);
```

Lekin current move se next player calculate kiya ja sakta hai:

```
1 const xIsNext = currentMove % 2 === 0;
```

Reason:

```
1 Move 0 → X
2 Move 1 → O
3 Move 2 → X
4 Move 3 → O
```

Even move par X, odd move par O.

Isliye xIsNext ko separate state mein rakhna redundant hai.

Important principle

Agar koi value existing props ya state se calculate ki ja sakti hai, to generally uski separate state banane ki zarurat nahi hoti.

Isse:

- State synchronization bugs kam hote hain
 - Code simpler hota hai
 - Source of truth clear rehta hai
-

Final code ka complete data flow

Jab user Square click karta hai:

```
1 1. Square ka button click hota hai
2 2. Square onClick call karta hai
3 3. Board ka arrow function handleClick(index) call karta hai
4 4. Board check karta hai:
5   - Winner already hai?
6   - Square already filled hai?
7 5. squares ki copy banti hai
8 6. X ya O set hota hai
9 7. Board onPlay(nextSquares) call karta hai
10 8. Game handlePlay receive karta hai
11 9. New board history mein add hota hai
12 10. currentMove latest move par set hota hai
13 11. Game re-render hota hai
14 12. Board updated squares receive karta hai
15 13. Square updated value display karta hai
```

CSS ka role

Universal selector

```
1 * {
2   box-sizing: border-box;
3 }
```

Element ki declared width aur height mein padding aur border ko include karta hai.

Square styling

```
1 .square {
2   background: #fff;
3   border: 1px solid #999;
4   float: left;
5   font-size: 24px;
6   font-weight: bold;
7   height: 34px;
8   width: 34px;
9   text-align: center;
10 }
```

Har button ko fixed square appearance deta hai.

Row clear karna

```
1 .board-row:after {  
2   clear: both;  
3   content: '';  
4   display: table;  
5 }
```

Floated Squares ke baad row layout ko clear karta hai.

Game layout

```
1 .game {  
2   display: flex;  
3   flex-direction: row;  
4 }
```

Board aur history list ko horizontal direction mein rakhta hai.

```
1 .game-info {  
2   margin-left: 20px;  
3 }
```

History list aur board ke beech spacing deta hai.

Tutorial se seekhe gaye main React concepts

1. Components

UI ko reusable pieces mein divide karna:

```
1 Game  
2 Board  
3 Square
```

2. JSX

JavaScript ke andar HTML-like UI syntax.

3. Props

Parent se child tak data aur callback pass karna:

```
1 <Square value={value} onSquareClick={callback} />
```

4. State

Component ko render ke beech information remember karana:

```
1 const [history, setHistory] = useState(...);
```

5. Event handling

User actions handle karna:

```
1 onClick={onSquareClick}
```

6. Lifting state up

Shared state ko common parent mein move karna.

7. Immutability

Old data modify karne ke bajay new copy create karna:

```
1 const nextSquares = squares.slice();
```

8. One-way data flow

Data parent se child jata hai. Child callback ke through parent ko changes notify karta hai.

9. Rendering lists

map() se arrays ko JSX elements mein convert karna.

10. Keys

Dynamic list items ko stable identity dena:

```
1 <li key={move}>
```

11. Derived state

Existing state se value calculate karna:

```
1 const xIsNext = currentMove % 2 === 0;
```

Sabse important mental model

Is poore tutorial ko ek simple rule se samjho:

State ko us component mein rakho jise us data par control chahiye, phir data props se neeche bhejo aur events callbacks se upar bhejo.

Final structure:

```

1  Game
2  |
3  | history, currentMove
4  |
5  └─ Board
6      | current squares, next player
7      |
8      └─ Square 0
9      └─ Square 1
10     └─ Square 2
11     └─ Square 3
12     └─ Square 4
13     └─ Square 5
14     └─ Square 6
15     └─ Square 7
16     └─ Square 8

```

Game single source of truth hai. Board game rules aur presentation coordinate karta hai.
Square simple reusable button hai.

Practice improvements

Tutorial ke end mein ye exercises di gayi hain:

1. Current move ke liye button ke badle You are at move #... dikhana.
2. Hardcoded Squares ko loops se generate karna.
3. Moves ko ascending/descending sort karne ka toggle banana.
4. Winning three Squares highlight karna aur draw message show karna.
5. History mein har move ki row aur column location dikhana.

Recommended learning order:

```

1  Current move label
2      ↓
3  Board loops
4      ↓
5  History sorting
6      ↓
7  Draw + winning highlight
8      ↓
9  Move coordinates

```

External links mentioned in the provided content

Mozilla/MDN links ko list mein include nahi kiya gaya hai.

- <https://nodejs.org/en/>
- <https://chrome.google.com/webstore/detail/react-developer-tools/fmkadmapgofadopljbjfkapdkoienihi?hl=en>
- <https://addons.mozilla.org/en-US/firefox/addon/react-devtools/>
- <https://microsoftedge.microsoft.com/addons/detail/react-developer-tools/gpphkfbcpiddadnkolpfcckpihlkkil>